

MEDIFLOW AI

Final Project Baseline & Technical Report

Project: MediFlow AI

Purpose: AI-Assisted Healthcare Administrative Intake

Technology: C++, CMake, libcurl, nlohmann/json, LLM API

Project Type: Hackathon **Frontier Engineering Challenge 2026**

1. Executive Summary

MediFlow AI is an AI-powered administrative intake system designed to reduce the repetitive workload involved in the first stage of patient registration.

Healthcare receptionists and administrative staff often spend valuable time asking the same questions, entering patient information, checking required fields, organizing documents, and routing patients to the appropriate administrative service.

MediFlow AI addresses this bottleneck through a modular pipeline of specialized AI agents.

The system collects information from the patient, transforms the conversation into structured data, validates the information, and verifies the result before presenting it to healthcare staff.

The primary objective is not to replace healthcare professionals or make medical decisions. Instead, MediFlow AI is designed to reduce repetitive administrative work so healthcare professionals can focus more of their time on patient care.

2. The Problem

The first interaction between a patient and a healthcare facility frequently contains repetitive administrative tasks.

A receptionist may need to:

- Ask for the patient's name.
- Ask for age or date of birth.
- Ask for the reason for the visit.
- Ask how long the problem has been present.
- Request a telephone number.
- Check required documents.
- Register the information.
- Identify missing information.
- Determine the appropriate administrative department.
- Forward the patient to the next stage.

The process can look like this:

Patient arrives
↓
Receptionist asks questions
↓
Information is entered manually
↓
Documents are checked
↓
Missing information is identified
↓
Patient is registered
↓
Patient is routed to the appropriate service

This process is repetitive and consumes time that could otherwise be used to support patients.

3. The Administrative Bottleneck

The bottleneck becomes more visible when the number of patients increases.

Assume that a receptionist spends an average of **8 minutes** processing one patient.

If a clinic receives **50 patients**, the total administrative workload is:

$$50 \times 8 \text{ minutes} = 400 \text{ minutes}$$

Converting this into hours:

$$400 \div 60 = 6.67 \text{ hours}$$

Therefore:

$$50 \text{ patients} \times 8 \text{ minutes} = \text{approximately 6 hours and 40 minutes}$$

This means that processing only 50 patients can represent almost a full working day of administrative intake.

Example

Number of Patients	Time per Patient	Total Time
10	8 minutes	80 minutes
20	8 minutes	160 minutes
30	8 minutes	240 minutes

Number of Patients	Time per Patient	Total Time
40	8 minutes	320 minutes
50	8 minutes	400 minutes / 6h 40m
100	8 minutes	800 minutes / 13h 20m

This illustrates the central problem addressed by MediFlow AI:

The more patients arrive, the more administrative time is consumed by repetitive information collection.

4. MediFlow AI Solution

MediFlow AI automates the initial administrative information collection process.

Instead of requiring a staff member to manually collect every piece of information, the patient can interact with the system through a computer, tablet, or future web/mobile interface.

For example:

MediFlow AI:

"Hello. What is the reason for your visit today?"

Patient:

"I want to schedule an appointment because I have had a headache for two days."

The system processes the conversation and extracts administrative information.

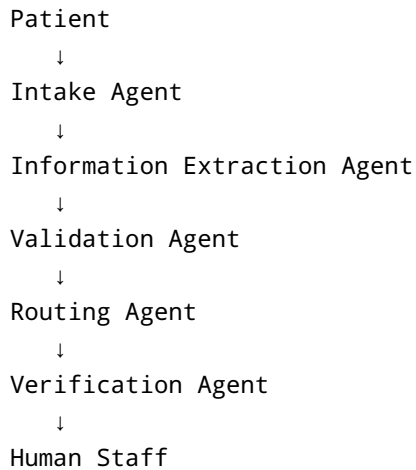
The resulting information can be transformed into structured data such as:

```
{
  "name": "João Manuel",
  "age": 34,
  "reason": "headache",
  "duration": "2 days",
  "phone": "923000000",
  "documents": []
}
```

The important principle is that the system does not attempt to diagnose the patient.

5. MediFlow AI Pipeline

The proposed workflow is:



The human healthcare worker remains responsible for clinical decisions.

MediFlow AI prepares and organizes the administrative information before it reaches the professional.

6. AI Agents

6.1 Intake Agent

The Intake Agent represents the first stage of the interaction.

Its responsibilities include:

- Collecting information from the patient.
- Asking administrative questions.
- Displaying the information provided by the patient.
- Identifying information that may need to be collected.

The Intake Agent does not diagnose the patient.

6.2 Information Extraction Agent

The Information Extraction Agent converts unstructured patient communication into structured data.

For example:

"My name is João. I am 34 years old and I have had a headache for two days."

can become:

```
{  
  "name": "João",  
  "age": 34,  
  "reason": "headache",  
  "duration": "2 days"  
}
```

This structured representation allows the C++ backend to process patient information more reliably.

6.3 Validation Agent

The Validation Agent checks whether required information is present and correctly formatted.

For example:

Name	✓
Age	✓
Reason	✓
Phone	✗

If a required field is missing, the validation result becomes invalid.

The current implementation verifies fields such as:

- Name
- Age
- Reason
- Phone

It also checks JSON types before attempting to read the values.

6.4 Document Agent

The Document Agent is part of the planned architecture.

Its future responsibilities include:

```
Document  
↓  
OCR  
↓  
Document Agent  
↓  
Information Extraction
```

↓
Validation

It could identify whether a required document is present and whether relevant information can be extracted.

This component was not fully implemented in the current MVP because of the limited development time available.

6.5 Routing Agent

The Routing Agent is intended to determine the appropriate administrative destination based on rules configured by the healthcare organization.

For example:

General Consultation → General Medicine

Dental Consultation → Dentistry

Eye Consultation → Ophthalmology

This is administrative routing and must not be presented as medical diagnosis or clinical decision-making.

6.6 Verification Agent

The Verification Agent provides an additional control layer.

Before the patient information is delivered to healthcare staff, the system can verify:

- Are the required fields complete?
- Is the information consistent?
- Did the AI invent information?
- Is each field supported by the patient's input?
- Did the previous validation stage succeed?

The conceptual flow is:

All Agents
↓
Verification Agent
↓
Information Complete?
↓

Information Supported?

↓

YES → Human Staff

NO → Additional Information Required

This verification layer is one of the main differentiating concepts of the MediFlow AI architecture.

The project principle is:

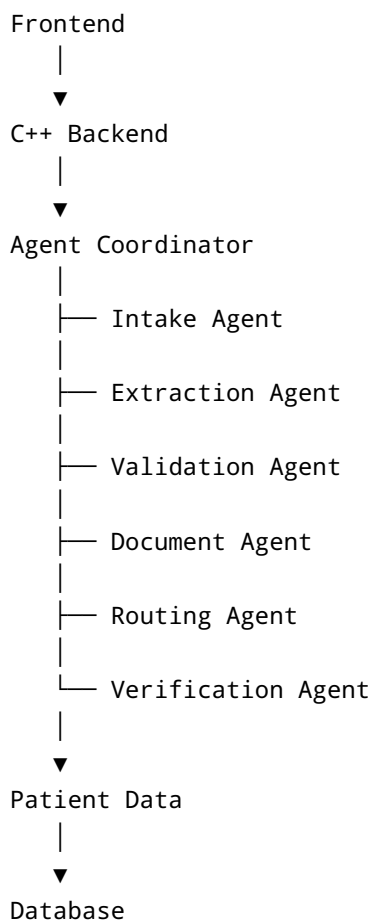
The AI should not simply generate information. It should verify the information before it reaches the healthcare worker.

7. C++ Architecture

C++ is used as the core backend technology.

The project is structured around independent components.

A simplified architecture is:



The database and frontend are part of the planned full architecture and were not the primary focus of the current MVP.

8. Current C++ Project Structure

The project follows a modular structure similar to:

```
mediflow/
├── src/
│   ├── main.cpp
│   ├── agents/
│   │   ├── Agent.hpp
│   │   ├── IntakeAgent.cpp
│   │   ├── ExtractionAgent.cpp
│   │   ├── ValidationAgent.cpp
│   │   ├── DocumentAgent.cpp
│   │   ├── RoutingAgent.cpp
│   │   └── VerificationAgent.cpp
│   ├── core/
│   │   └── MediFlowPipeline.cpp
│   ├── models/
│   │   └── Patient.cpp
│   └── llm/
│       └── LLMClient.cpp
├── include/
├── tests/
├── data/
├── CMakeLists.txt
└── README.md
```

The current implementation already contains the core pipeline components.

9. Core C++ Components

Agent

The base Agent class provides a common interface:

```
class Agent
{
public:
    virtual ~Agent() = default;

    virtual std::string process(
        const std::string& input
    ) = 0;
};
```

This allows different agents to share the same interface while implementing different responsibilities.

10. Patient Model

The Patient model represents structured patient information.

The current model contains:

```
struct Patient
{
    int age = 0;
    std::string name;
    std::string reason;
    std::string duration;
    std::string phone;

    std::vector<std::string> documents;
};
```

The system also provides conversion between the C++ object and JSON:

```
Patient → JSON
JSON → Patient
```

This makes it possible for the LLM output to be converted into a C++ data structure.

11. LLMClient

The `LLMClient` is responsible for communication with the language model API.

Its interface is:

```
class LLMClient
{
public:
    LLMClient();

    std::string chat(
        const std::string& systemPrompt,
        const std::string& userMessage
    );

private:
    std::string apiKey;
    std::string model;
};
```

The communication flow is:

```
C++
↓
LLMClient
↓
libcurl
↓
LLM API
↓
JSON Response
↓
C++
```

The API key is obtained through an environment variable rather than being hard-coded into the source code.

The model can also be configured externally.

12. JSON Communication

JSON is used as the primary communication format between the LLM and the C++ backend.

Example:

```
{
  "name": "João Manuel",
  "age": 35,
  "reason": "headache",
  "duration": "three days",
  "phone": "923000000",
  "documents": []
}
```

After validation, additional information can be attached:

```
{
  "name": "João Manuel",
  "age": 35,
  "reason": "headache",
  "duration": "three days",
  "phone": "923000000",
  "documents": [],
  "validation": {
    "valid": true
  }
}
```

The Verification Agent can then produce:

```
{
  "verification": {
    "approved": true
  }
}
```

This structured approach makes the communication between components predictable and easier to debug.

13. Current MediFlow Pipeline

The current pipeline is implemented through the `MediFlowPipeline` class.

The processing sequence is:

```
1. Intake
   ↓
```

2. Extraction
- ↓
3. Validation
- ↓
4. Verification
- ↓
5. Result

The pipeline first receives the patient's input.

The Intake Agent processes the initial information.

The Extraction Agent communicates with the LLM and produces structured information.

The Validation Agent verifies required fields.

The Verification Agent checks whether the information has successfully passed validation.

Finally, the system can produce an approved administrative patient record.

14. MVP Demonstration

The current MVP can demonstrate the following scenario:

```
=====
                MEDIFLOW AI
=====

Patient verified successfully

Name: João Manuel
Age: 35
Reason: headache
Duration: three days
Phone: 923000000

Status: APPROVED

=====
```

This demonstrates the core value proposition:

Raw patient input → Structured information → Validation → Verification → Approved result

15. Performance Concept

The main performance objective is to reduce the amount of administrative time required per patient.

Using the previous example:

Traditional process

50 patients × 8 minutes
= 400 minutes
= 6 hours 40 minutes

As a theoretical target, if MediFlow reduced the administrative interaction to approximately 2 minutes:

50 patients × 2 minutes
= 100 minutes
= 1 hour 40 minutes

The theoretical difference would be:

400 - 100
= 300 minutes
= 5 hours

The theoretical percentage reduction would be:

$300 \div 400 \times 100$
= 75%

Therefore:

Traditional: 6h 40m
MediFlow Target: 1h 40m

Potential difference: 5 hours
Potential reduction: 75%

Important: The 2-minute figure and 75% reduction are target/example values. They must not be presented as measured project results unless they are confirmed through an actual benchmark.

16. Evaluation Method

A proper evaluation should use fictional but representative patient cases.

For example, 30 cases can be processed using both approaches.

Traditional Process

Measure:

- Average time per patient.
- Number of missing fields.
- Number of incorrectly entered fields.
- Number of administrative errors.

MediFlow Process

Measure:

- Average processing time.
- Missing required fields.
- Extraction errors.
- Validation results.
- Verification results.

The final benchmark can then compare the two approaches.

Example:

Metric	Traditional	MediFlow
Average processing time	Measured	Measured
Missing fields	Measured	Measured
Administrative errors	Measured	Measured
Verification accuracy	N/A	Measured

The final presentation should use real measurements instead of hypothetical numbers.

17. Technology Stack

Current Technologies

Backend

- C++
- C++17/C++20

- CMake
- libcurl

Data Processing

- nlohmann/json
- Structured JSON communication
- C++ data models

Artificial Intelligence

- LLM API
- Groq API integration
- Configurable model

Architecture

- Object-oriented design
 - Polymorphic Agent interface
 - Modular agent pipeline
-

18. Planned Technologies

The full production architecture can later include:

REST API

A C++ HTTP framework such as:

- Crow
- Drogon

Database

Potential database technologies:

- SQLite
- PostgreSQL

Frontend

Potential technologies:

- HTML
- CSS
- JavaScript
- React

Document Processing

- OCR
- Document classification
- Document information extraction

Deployment

- Cloud infrastructure
 - Containerization
 - CI/CD
 - Production monitoring
-

19. Current MVP vs Future System

The project was intentionally developed in stages.

Implemented / Core MVP

```
graph TD;
  A[Patient Input] --> B[Intake Agent];
  B --> C[Extraction Agent];
  C --> D[LLM];
  D --> E[Structured JSON];
  E --> F[Validation Agent];
  F --> G[Verification Agent];
  G --> H[Approved Result];
```

Future Complete System

```
graph TD;
  A[Web / Mobile Interface] --> B[REST API];
  B --> C[C++ Backend];
  C --> D[Agent Coordinator];
  D --> E[ ];
```




20. Development Constraints

The project was developed under a strict hackathon deadline.

Because of the limited time available, development priorities were focused on demonstrating the central technical concept rather than implementing the entire production architecture.

The priority was therefore:

1. Build the C++ architecture.
2. Implement the Agent abstraction.
3. Implement patient data structures.
4. Integrate the LLM.
5. Implement structured JSON extraction.
6. Implement validation.
7. Implement verification.
8. Create an executable demonstration.

Additional components such as a complete frontend, persistent database, OCR infrastructure, production API and deployment were left for future development.

This was a deliberate prioritization decision rather than an architectural limitation.

21. Future Development

The next development stage can include:

Frontend

A web or tablet interface where patients interact directly with MediFlow.

Database

Persistent storage for patient administrative information.

REST API

A C++ backend API connecting the frontend to the agent pipeline.

Document Agent

OCR-based document processing and validation.

Routing Agent

Configurable administrative routing rules.

Healthcare Dashboard

A dashboard where staff can view verified patient intake records.

Authentication

User authentication and authorization for healthcare staff.

Testing

Automated unit, integration and system tests.

Deployment

Cloud-based deployment and monitoring.

22. Security and Safety Principles

MediFlow AI must operate within a clearly defined scope.

The system is not intended to:

- Diagnose diseases.
- Prescribe medication.
- Replace doctors.
- Replace nurses.
- Make clinical decisions.

Its role is administrative information collection and organization.

The human healthcare professional remains responsible for clinical decisions.

The Verification Agent is intended to reduce the risk of unsupported or incorrectly structured information reaching the healthcare worker.

23. Main Innovation

The project is not simply an AI chatbot.

The main idea is the combination of:

Conversational AI
+
Structured Data
+
Validation
+
Verification
+
Human Oversight

The Verification Agent is particularly important because the system does not stop after generating information.

It introduces another stage that asks whether the collected information is complete, valid and supported by the patient's input.

This creates a stronger agent-based architecture.

24. Key Value Proposition

The main value proposition can be summarized as:

MediFlow AI reduces repetitive administrative work during patient intake by collecting, structuring, validating and verifying patient information before it reaches healthcare staff.

The goal is not to replace healthcare workers.

The goal is to give them more time to focus on healthcare.

25. Final Hackathon Pitch

Healthcare workers spend valuable time collecting and organizing information that patients could provide themselves.

If a receptionist spends only eight minutes processing each patient, 50 patients represent:

$50 \times 8 = 400$ minutes

That's approximately:

6 hours and 40 minutes of administrative work.

MediFlow AI is designed to reduce this bottleneck.

Our system uses specialized AI agents to collect patient information, transform conversations into structured data, validate the required fields and verify the information before it reaches healthcare staff.

Our current MVP focuses on the core C++ AI pipeline.

Because of the limited hackathon development time, components such as the complete frontend, database, OCR system, REST API and production infrastructure are planned for future development.

MediFlow AI does not diagnose patients.

It prepares structured and verified administrative information so healthcare professionals can focus on what matters most:

providing healthcare.

26. Conclusion

MediFlow AI demonstrates how C++ and modern AI technologies can be combined to address a practical healthcare administration problem.

The project focuses on a specific bottleneck: repetitive patient intake.

At eight minutes per patient, 50 patients can represent approximately six hours and forty minutes of administrative work.

By automating the collection and organization of information, MediFlow AI aims to reduce this workload.

The current MVP demonstrates the fundamental technology:

```
graph TD
    Patient --> Intake
    Intake --> Extraction
    Extraction --> 
```

```
graph TD; Validation --> Verification; Verification --> HumanStaff[Human Staff];
```

Validation
↓
Verification
↓
Human Staff

The architecture was intentionally designed to be extensible.

Future versions can add a web interface, database, REST API, OCR, document processing, routing, authentication, dashboards and production deployment without fundamentally changing the core agent architecture.

The central philosophy of MediFlow AI is simple:

We are not replacing healthcare professionals. We are reducing the repetitive administrative work that keeps them away from patients.